Tabla de contenido

1. Introducción.....3
2. Suposiciones y Prerrequisitos del Entorno.....3
3. Descripción de la Solución y Decisiones de Desarrollo.....5
4. Funcionamiento del Script.....7
Fase 1: Aprovisionamiento de Hardware desde el Host.....7
Fase 2: Sincronización y Validación de Comunicación.....8
Fase 3: Configuración Interna en el Sistema Operativo Invitado.....8
Planteamiento del Script.....9
5. Escenario de Aplicación.....11
6. Principales Problemas Encontrados y Soluciones.....12
6.1 Problemas con la configuración del archivo psd1.....12
6.2 No se encuentra la máquina virtual.....13
6.3 Conector equivocado.....14
6.4 Error en la ejecución de comandos dentro de la máquina virtual.....15
6.5 Problema con permisos de sudo.....17
6.6 Problema en identificación del disco.....17
7. Diseño de Pruebas y Resultados.....19
7.1 Validación de configuración.....19
7.1 Archivo psd1 no existente.....19
7.2 Atributo faltante en el psd1.....20
7.3 configuración inicial de la VM 1.....22
7.4 Configuración de una segunda VM.....23
7.5 Disponibilidad del almacenamiento.....25
7.6 Persistencia de los datos tras reiniciar la máquina virtual.....27
7.7 Ejecución repetida sobre una VM ya configurada.....28
7.8 Ejecución con la máquina virtual encendida.....30
8. Evidencias ejecución correcta.....31
Tiempos de ejecución de algunas pruebas:.....34
Herramientas de IA.....35
Parámetros de la máquinas virtuales.....35
9. Referencias.....36

1. Introducción

La administración eficiente de infraestructuras virtualizadas requiere minimizar la intervención manual en tareas repetitivas, como el aprovisionamiento y configuración de almacenamiento. Por lo anterior, el presente documento detalla el desarrollo de una solución de automatización que permite aprovisionar y configurar almacenamiento adicional de datos para una máquina virtual GNU/Linux, ejecutando el proceso de manera íntegra desde el sistema operativo anfitrión (host).

El equipo de trabajo cuenta con experiencia técnica previa, adquirida en prácticas anteriores donde se realizaba manualmente la preparación del disco, la creación de particiones y sistemas de archivos, y la configuración del montaje automático; este desarrollo avanza hacia la implementación de un mecanismo completamente automatizado, reproducible y reutilizable para múltiples máquinas virtuales. Además se hace uso de las capacidades de Oracle VM VirtualBox y empleando Guest Additions como puente de interacción entre el host y el sistema operativo invitado, la solución desarrollada logra conectar y preparar el almacenamiento sin intervención del usuario durante su ejecución.

A lo largo de este documento se expondrá el escenario de pruebas, el funcionamiento lógico del script, las decisiones técnicas adoptadas y las pruebas sistemáticas ejecutadas. Estas pruebas demuestran la eficacia de la solución, su capacidad para gestionar variaciones entre distintas máquinas virtuales y su comportamiento seguro para evitar la pérdida de datos ante ejecuciones repetidas.

2. Suposiciones y Prerrequisitos del Entorno

Con el fin de garantizar el correcto funcionamiento del script de automatización y la ejecución exitosa de los comandos remotos, el equipo de desarrollo asume las siguientes condiciones previas en el entorno de virtualización:

- **Existencia de las Máquinas Virtuales:** Las máquinas virtuales que actuarán como destino deben estar previamente creadas en el Oracle VirtualBox del equipo host. El script no aprovisiona la máquina desde cero, únicamente el almacenamiento adicional.
- **Instalación de Guest Additions:** Las herramientas de Guest Additions deben estar instaladas, actualizadas y en ejecución dentro del sistema operativo invitado. Este componente es indispensable para habilitar el canal de comunicación mediante VBoxManage guestcontrol.
- **Configuración de Red en Modo Puente:** La máquina guest debe tener habilitado y conectado su primer adaptador de red configurado en modo puente. Esto permite que la máquina obtenga una dirección IP válida y accesible en la misma red local que el equipo host.
- **Sistema Operativo Invitado GNU/Linux:** El sistema operativo ejecutándose en las máquinas virtuales debe ser una distribución basada en GNU/Linux. La lógica de configuración y los comandos inyectados por el script como parted, mkfs.ext4, lsblk, findmnt y la edición de /etc/fstab; son exclusivos de este sistema.
- **Privilegios Administrativos:** El usuario y la contraseña proporcionados en el archivo de configuración externo, deben corresponder al usuario root o a un usuario con privilegios de sudo configurado para no requerir contraseña interactiva, dado que operaciones como formatear o montar discos requieren permisos.
- **Controlador de Almacenamiento SATA:** Se asume que la máquina virtual tiene configurado un controlador de almacenamiento SATA (tipo IntelAhci) con al menos un puerto libre donde el script pueda conectar dinámicamente el disco virtual.
- **Disponibilidad de VBoxManage en el Host:** El equipo host debe contar con Oracle VM VirtualBox instalado y la herramienta de línea de comandos VBoxManage debe estar accesible en las variables de entorno del sistema operativo para que PowerShell pueda invocarla sin errores.
- **Máquina virtual apagada al iniciar:** Antes de ejecutar el script, la máquina virtual debe encontrarse apagada. El script no la apaga por su cuenta; si detecta

que está encendida, se detiene con el código de salida 14 y no realiza ningún cambio.

- **Archivo de configuración completo y ruta relativa:** El archivo config\vm.psd1 debe incluir todos los atributos (VmName, GuestUser, Pass, DiskSizeMb, MountPoint, FsLabel, VdiPath y UseSudo). Además, el script debe ejecutarse desde su propia carpeta, porque la ruta del archivo de configuración es relativa.
- **Entorno de ejecución en el equipo anfitrión:** El equipo debe ser Windows con PowerShell 5.0 o superior, versión que incorpora Import-PowerShellDataFile, y con una política de ejecución que permita correr scripts.

3. Descripción de la Solución y Decisiones de Desarrollo

La solución desarrollada por el equipo de trabajo consiste en un script implementado en PowerShell que automatiza la creación, asignación y configuración interna de un disco virtual para almacenamiento de datos en máquinas virtuales GNU/Linux. El diseño del script permite que el proceso completo se ejecute desde el sistema operativo host sin que el usuario tenga que ingresar información, escribir comandos manualmente, introducir contraseñas o presionar teclas durante la ejecución; justo como se solicita.

Para garantizar que la solución sea reutilizable y pueda aplicarse a diferentes máquinas virtuales, se tomó la decisión de desarrollo de no codificar (hardcodear) los valores dentro del script. En su lugar, se implementó la lectura de un archivo de configuración externo con formato de diccionario (.\config\vm.psd1). Este archivo almacena los parámetros que pueden variar entre las máquinas, tales como el nombre de la VM, el tamaño del disco, las credenciales del sistema invitado y el punto de montaje. El script valida que este archivo exista; si falta, muestra el error en pantalla y termina con el código de salida 2, antes de realizar cualquier operación sobre la máquina virtual, lo cual permite controlar escenarios y comportamientos ante condiciones que puedan impedir la ejecución normal del proceso. En este punto no se genera registro (log), porque todavía no ha sido creado; los errores posteriores sí quedan registrados.

Antes de modificar la configuración de VirtualBox, el script realiza comprobaciones críticas de entorno. Se asegura de que la máquina destino posea un adaptador de red en modo puente y valida que la máquina virtual se encuentre apagada. Superadas las validaciones, el script aprovisiona automáticamente el disco virtual, conectándolo al controlador SATA sin

requerir uso de la interfaz gráfica. Esto se realizó con el fin de cumplir los requerimientos descritos en el documento guía.

Una vez aprovisionado el disco, la máquina virtual es iniciada y el script monitorea dinámicamente la obtención de la dirección IP. Posteriormente, se establece la comunicación entre el host y el sistema operativo invitado empleando el mecanismo de Guest Additions (VBoxManage guestcontrol). A través de esta vía de comunicación, el orquestador envía comandos de manera no interactiva para ejecutar las operaciones necesarias de particionamiento, creación del sistema de archivos, montaje y configuración del montaje automático en el archivo fstab.

Durante el desarrollo se tomó la decisión de implementar idempotencia en los comandos internos, cumpliendo así con el requerimiento de que una segunda ejecución no provoque la pérdida de los datos almacenados ni realice operaciones destructivas. Para lograr esto, la solución ejecuta validaciones paso a paso en el sistema operativo Linux: Identifica el dispositivo y verifica si ya existe una partición; de no existir, se crea una partición utilizando todo el espacio disponible. Utiliza herramientas como blkid para evaluar si la partición ya cuenta con un sistema de archivos antes de proceder a formatearla en formato Ext4. Verifica la existencia del punto de montaje en el sistema de archivos, antes de invocar la creación del directorio. Comprueba con findmnt si el almacenamiento ya está montado y busca el UUID del disco en /etc/fstab para saber si ya posee una entrada. Si la entrada ya existe, la solución omite el paso, evitando duplicarla.

Finalmente, para facilitar la obtención de tiempos de ejecución y registrar el comportamiento del script, se decidió implementar un sistema de telemetría propio. Además de generar un archivo .log por cada ejecución con la traza completa de comandos, el script calcula el tiempo que transcurre desde el encendido de la máquina hasta la obtención de red, el tiempo desde ese momento hasta obtener disponibilidad de Guest Control, y el tiempo total de la ejecución. Estas métricas, junto con un veredicto de estado (OK o FALLIDO por algún subpaso), se concatenan automáticamente en un archivo tiempos.csv. Este diseño permite ejecutar la herramienta repetidas veces sobre múltiples máquinas virtuales y tener un registro centralizado y comparativo de los resultados para su posterior análisis.

Otras decisiones de diseño complementan lo anterior. Antes de cualquier cambio, el script también comprueba que VBoxManage (la herramienta de línea de comandos de VirtualBox) esté disponible y que la máquina esté registrada. El script no apaga la máquina por su cuenta; si la encuentra encendida, se detiene con el código de salida 14, para no interrumpir procesos que pudieran estar en ejecución. Se eligió Guest Control en lugar de SSH porque no depende de una sesión activa ni de configurar claves, y los comandos internos se envían de forma directa, sin un script intermedio en bash. Todos se ejecutan con sudo -n; la opción -n hace que, si sudo solicitara contraseña, falle en lugar de quedarse esperando, con lo que el proceso nunca se detiene a esperar que alguien escriba algo. Cada tipo de error termina con un código de salida distinto (por ejemplo 2, 14 o 30), lo que facilita identificar la causa. Al finalizar, la máquina virtual queda encendida. La fila de tiempos.csv solo se escribe cuando el proceso llega hasta el final; las detenciones tempranas, como la máquina encendida o el archivo de configuración ausente, quedan únicamente en el registro (log) o en pantalla.

4. Funcionamiento del Script

El script opera bajo una arquitectura de ejecución secuencial que transiciona desde la administración de hardware virtualizado en el sistema host hasta la ejecución de comandos a nivel de sistema operativo en la VM. Antes de iniciar cualquier interacción con el hipervisor, el script lee un archivo de configuración en formato de datos de PowerShell (.\config\vm.psd1). Este archivo opera como una tabla de tipo clave-valor que centraliza los parámetros que pueden variar en diferentes escenarios de aplicación, permitiendo que la lógica principal de la solución no dependa de un único entorno. En su interior, se definen variables como: VmName (nombre exacto de la máquina destino), DiskSizeMb (capacidad requerida para el disco virtual), GuestUser y Pass (usuario y contraseña necesarios para autenticar las sesiones remotas), MountPoint (ruta absoluta del directorio en el sistema de archivos Linux donde se montará el disco), FsLabel (etiqueta asignada al sistema de archivos), VdiPath (ruta física en el host para el archivo .vdi) y UseSudo (indica si los comandos internos se ejecutan con privilegios de administrador mediante sudo -n).

Este flujo se divide en tres fases principales: aprovisionamiento, sincronización y configuración interna.

Fase 1: Aprovisionamiento de Hardware desde el Host

La ejecución comienza validando que VBoxManage esté disponible, que la máquina virtual esté registrada, que su primer adaptador de red esté en modo puente y que se encuentre apagada. A continuación, el script gestiona el almacenamiento físico interactuando con el hipervisor a través de VBoxManage. Primero, inspecciona el controlador de almacenamiento (buscando el tipo IntelAhci) y localiza un puerto SATA disponible a partir del puerto 1. Si el archivo .vdi especificado no existe en el disco del host, se crea dinámicamente utilizando el tamaño proveído en la configuración, y finalmente se adjunta a la máquina virtual.

Fase 2: Sincronización y Validación de Comunicación

Una vez conectado el almacenamiento, la máquina virtual se inicia en modo headless. Debido a que el sistema operativo invitado tarda un tiempo variable en iniciar sus servicios, el script implementa un mecanismo de sondeo periódico. El script consulta iterativamente la propiedad /VirtualBox/GuestInfo/Net/0/V4/IP hasta obtener la dirección IP asignada por el adaptador en modo puente; si transcurren 300 segundos sin obtenerla, el proceso termina con el código de error 20. Sin embargo, obtener una IP no garantiza que el servicio de Guest Additions esté listo para recibir comandos. Para confirmar la comunicación entre el host y la máquina virtual, el script envía un comando de prueba simple junto con las credenciales. Solo cuando este comando retorna un código de éxito, el script asume que el sistema operativo está completamente funcional para comenzar la configuración; si no lo logra en 300 segundos, termina con el código 21.

Fase 3: Configuración Interna en el Sistema Operativo Invitado

Toda la configuración interna se orquesta mediante una función envolvente que ejecuta comandos de GNU/Linux de forma remota usando VBoxManage guestcontrol run. El script procesa el resultado de cada comando para tomar decisiones en tiempo real:

-

- Identificación: Se utiliza lsblk -dn -o NAME,SIZE,TYPE para listar los dispositivos de bloque; se descarta el disco del sistema (sda) y cualquier dispositivo que no sea de tipo disk, como las unidades ópticas, y se selecciona el disco de datos restante.
- Particionamiento: Si el disco no contiene particiones, se invoca la herramienta parted para crear una tabla de particiones GPT (formato moderno de particionamiento) con una partición que ocupe todo el espacio.
- Formateo: Se verifica el formato actual con blkid. Si la partición no tiene sistema de archivos, se aplica el comando mkfs.ext4 para crear el sistema de archivos Ext4 asignando la etiqueta definida en la configuración.
- Punto de montaje y activación: Se verifica si la carpeta del punto de montaje existe y, de no ser así, se crea con mkdir -p. Luego, con findmnt se comprueba si el disco ya está montado; en caso contrario, se ejecuta el comando mount para que el almacenamiento quede disponible inmediatamente, sin necesidad de reiniciar.
- Montaje automático: Se extrae el identificador único de la partición (UUID) ejecutando blkid -s UUID -o value y se verifica con grep si existe en /etc/fstab. Si no se encuentra, se agrega la nueva configuración al final del archivo con sudo -n tee -a /etc/fstab, verificando que la escritura haya sido exitosa.

Durante la Fase 3, el script opera como una máquina de estados. Si en algún momento un comando de configuración falla (por ejemplo, al particionar o por permisos insuficientes), una variable de control registra en qué subpaso ocurrió la falla. Los subpasos siguientes se omiten para evitar que generen daños en el sistema de archivos; el script registra el estado en el archivo de métricas (tiempos.csv) y finaliza con el código de salida 30. Cada ejecución genera además un archivo .log con la traza completa de los comandos y sus respuestas.

Planteamiento del Script

1. Configuración externa (ruta fija, sin argumentos)

El script busca siempre .\config\vm.psd1.
Carga la tabla de valores (hashtable) con VmName, GuestUser, Pass, DiskSizeMb, MountPoint, FsLabel, VdiPath y UseSudo.
Si el archivo no existe, muestra el error en pantalla y termina con el código 2. Si GuestUser o

Pass faltan o están vacíos, muestra el error y termina con el código 3. En ambos casos el proceso se detiene antes de tocar la VM y no se genera registro (log), porque todavía no ha sido creado; son casos útiles para la prueba de "condición que impide la ejecución normal".

2. Preparación del registro

Crea logs\ si no existe.
Genera un log con el nombre de la VM (tomado del .psd1) y la fecha y hora.
Inicia el cronómetro del tiempo total.

3. Validación del entorno

VBoxManage está disponible (código 10).
La VM (VmName) existe en VirtualBox (código 11).
El adaptador de red 1 está en modo puente (código 13).
La VM está apagada (código 14). Si no lo está, se registra el fallo y el script termina; no la apaga por su cuenta.

4. Crea y conecta el disco de datos

Verifica si el .vdi ya está conectado a la VM.
Si no lo está, busca el controlador SATA (IntelAhci) y el primer puerto libre desde el puerto 1, crea el .vdi con DiskSizeMb si no existe y lo conecta. Si no hay controlador SATA, termina con el código 15; si falla la conexión, con el código 16.

5. Arranca la VM

Ejecuta startvm --type headless y guarda la marca de tiempo de inicio. Si falla, termina con el código 17.

6. Espera la IP

Consulta la propiedad GuestInfo/Net/0/V4/IP cada 2 segundos, hasta un máximo de 300 segundos (variable interna $TiempoMaximoEspera). Si se agota el tiempo, termina con el código 20.

7. Espera Guest Control

Envía el comando de prueba /bin/true con las credenciales de GuestUser y Pass, y reintenta cada 2 segundos hasta 300 segundos. Si nunca responde, termina con el código 21 y un mensaje que sugiere revisar usuario y contraseña.

8. Configuración del disco (comandos directos vía Guest Control, sin script intermedio)

Cada subpaso es una llamada independiente a VBoxManage guestcontrol run que ejecuta el comando con bash -c, anteponiendo sudo -n cuando UseSudo está activo. Cada llamada queda registrada en el log con su comando y su respuesta.

8.1 Identificar el disco. Ejecuta lsblk -dn -o NAME,SIZE,TYPE, descarta sda y los dispositivos que no son de tipo disk, y selecciona el último disco restante; no compara el tamaño. Si no

encuentra candidato, registra el error en el subpaso 8.1.
8.2 Verificar la partición. Ejecuta lsblk -n sobre el disco. Si ya existe una partición, registra que se omite; si no, la crea con parted (tabla GPT, una partición que ocupa todo el disco).
8.3 Verificar el sistema de archivos. Ejecuta blkid sobre la partición. Si ya devuelve un tipo (TYPE=), registra que se omite; si no, ejecuta mkfs.ext4 con la etiqueta FsLabel.
8.4 Verificar o crear el punto de montaje. Comprueba con test -d si existe MountPoint; si no, lo crea con mkdir -p. En ambos casos lo registra.
8.5 Verificar el montaje. Ejecuta findmnt sobre MountPoint. Si ya está montado, registra que se omite; si no, ejecuta mount.
8.6 Verificar o agregar la entrada de fstab. Obtiene el UUID con blkid -s UUID -o value y lo busca con grep en /etc/fstab. Si existe, registra que se omite; si no, agrega la línea con sudo -n tee -a y verifica que la escritura haya sido exitosa.

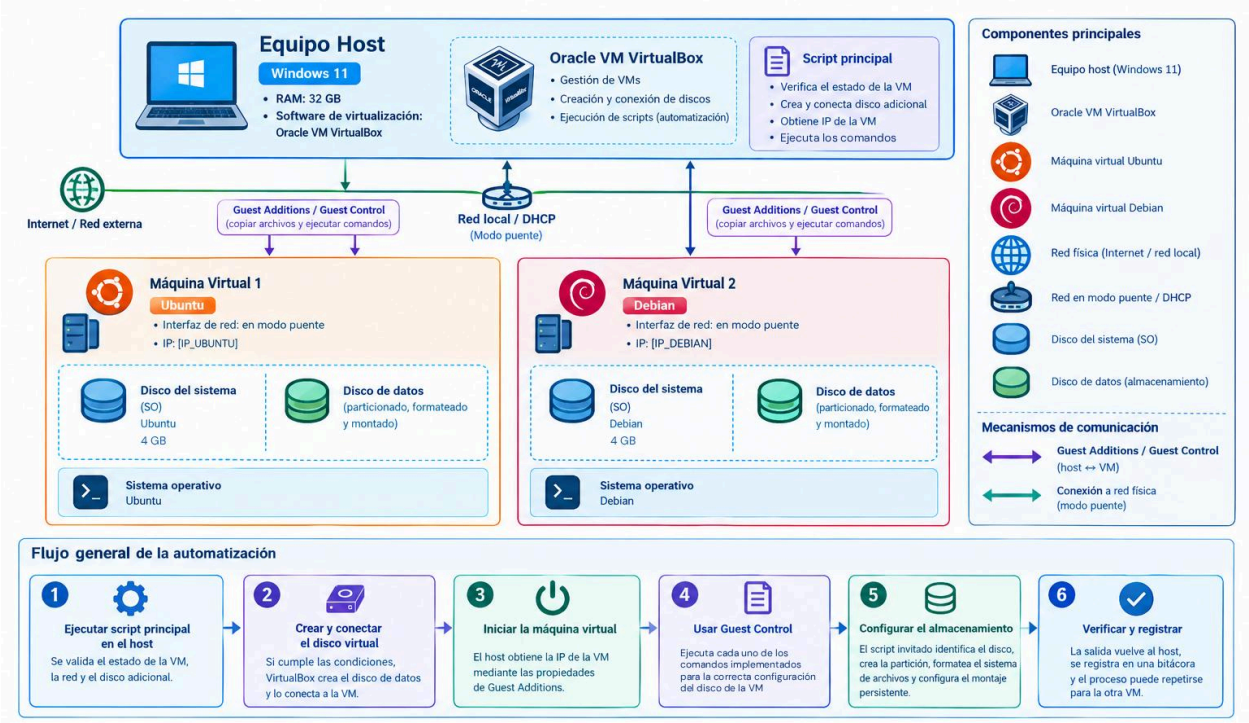
9. Veredicto final

Una variable de control guarda el subpaso donde ocurrió el primer fallo; a partir de ese momento, los subpasos siguientes se omiten.
Si ningún subpaso falló (ejecutados u omitidos por ya existir), el estado es OK.
Si alguno falló, el estado es FALLIDO en paso X. El script registra el estado, guarda las métricas y termina con el código 30.

10. Registro de métricas

Agrega una fila a tiempos.csv con la fecha, la VM, la IP, el tiempo hasta obtener IP (desde el encendido), el tiempo hasta Guest Control (desde la IP), el tiempo total y el estado. La fila solo se escribe si el proceso llega hasta este paso; las detenciones tempranas quedan únicamente en pantalla o en el log.
Al ser un archivo acumulativo, se puede ejecutar el script sobre distintas VM reemplazando el contenido de vm.psd1 entre corridas; así queda un historial comparativo para la sección "Tiempo de ejecución".

5. Escenario de Aplicación



Desde el equipo host (Windows 11) se ejecuta el script orquestador.ps1, encargado de automatizar la configuración del almacenamiento en dos máquinas virtuales GNU/Linux administradas con Oracle VM VirtualBox: ubuntu (192.168.20.94) y debian (192.168.20.95). Cada una tiene su propio disco de sistema (sda, de aproximadamente 30 GB en ubuntu y 21 GB en debian) y un disco virtual adicional de 4 GB destinado al almacenamiento de datos (ubuntu-srv.vdi y debian-srv.vdi).

Ambas máquinas utilizan el adaptador de red 1 en modo puente (bridged) sobre la misma interfaz física del host y obtienen su dirección IP mediante DHCP en la red 192.168.20.x. La comunicación para la automatización se realiza mediante las Guest Additions: el host obtiene la IP consultando la propiedad /VirtualBox/GuestInfo/Net/0/V4/IP con VBoxManage guestproperty y envía cada comando de configuración de forma directa con VBoxManage guestcontrol run, sin utilizar SSH y sin copiar ni ejecutar scripts dentro de la VM.

El proceso automatizado valida el entorno, crea y conecta el disco adicional, enciende la máquina, espera a que Guest Control esté disponible, identifica el dispositivo de almacenamiento dentro de la máquina virtual, lo particiona, crea el sistema de archivos ext4 y configura su montaje persistente en /almacen mediante una entrada de fstab basada en el

UUID. Finalmente, el host registra la traza completa en un archivo de log y los tiempos y el estado en tiempos.csv. La solución procesa una máquina a la vez; para configurar la otra, se reemplazan los valores de config\vm.psd1 y se ejecuta de nuevo el script.

6. Principales Problemas Encontrados y Soluciones

6.1 Problemas con la configuración del archivo psd1

Primer problema aunque no tenemos capturas de pantalla de la terminal, fue uno sencillo, al inicio teníamos el archivo psd1 con un atributo llamado PassFile, el cual tenía la ruta a un archivo con la contraseña de la máquina virtual objetivo. Esta decisión se tomó porque, al revisar diferentes formas de organizar la configuración, parecía una práctica recomendable mantener la contraseña separada del archivo principal y evitar que quedara escrita directamente en la configuración.

Sin embargo, durante las pruebas se presentaron errores relacionados con la ruta del archivo que contenía la contraseña; el script no lograba obtener correctamente el contenido del archivo y esto impedía que se estableciera la conexión mediante Guest Control. Debido a que el problema se presentaba de forma constante y la separación de la contraseña no estaba siendo necesaria para el funcionamiento planteado, se decidió simplificar este mecanismo.

Como solución, se reemplazó el atributo PassFile por un atributo Pass dentro del mismo archivo vm.psd1; de esta manera, el script obtiene directamente la contraseña desde la configuración al momento de ejecutar los comandos dentro de la máquina virtual. Esta modificación permitió eliminar los problemas asociados con las rutas de archivos externos y hacer más sencilla la configuración del proceso. En el archivo utilizado actualmente, el atributo Pass contiene directamente la contraseña que será utilizada junto con el usuario definido en GuestUser.

La solución permitió continuar con el desarrollo del proceso de automatización y verificar correctamente la disponibilidad de Guest Control; posteriormente, este mecanismo se utilizó en la función encargada de ejecutar los comandos dentro de la máquina virtual.

6.2 No se encuentra la máquina virtual

Por alguna razón no reconoce cual es la máquina virtual llamada ubuntu

```
PS C:\Users\mavar\OneDrive\Escritorio\nube> . 'C:\Users\mavar\OneDrive\Escritorio\nube\orquestador.ps1
[2026-09-27 16:20:19] Inicio del proceso para la VM: ubuntu
[2026-09-27 16:20:19] ERROR: la VM 'ubuntu' no existe en VirtualBox
PS C:\Users\mavar\OneDrive\Escritorio\nube>
```

Sin embargo, cuando listamos las máquinas virtuales, si nos aparece la máquina llamada ubuntu

```
C:\Users\mavar>VBoxManage list vms
"ubuntu" {8164718e-858a-4b7a-9e1e-fadaba088f14}
"debian" {04e9ad11-3df4-4ed1-ba51-9b78c5e02c43}

C:\Users\mavar>
```

Revisando cuál es el problema en el código es la forma en la que se obtienen las máquinas virtuales, se usa -notmatch para verificar, sin embargo este método no está diseñado para recibir una lista de máquinas virtuales, por eso mismo nunca encuentra ninguna.

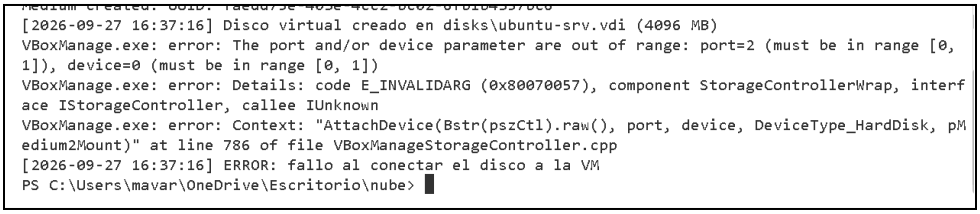
```
$ListaVMs = & $VBoxManage list vms
if ($ListaVMs -notmatch [regex]::Escape($Config.VmName)) {
    Escribir-Log "ERROR: la VM '$($Config.VmName)' no existe en VirtualBox"
    exit 11
}
```

La corrección que se propone para este error es buscar si alguna línea coincide con el nombre, de la siguiente manera:

```
$VmEncontrada = $ListaVMs | Where-Object { $_ -match [regex]::Escape($Config.VmName)
}
if (-not $VmEncontrada) {
    Escribir-Log "ERROR: la VM '$($Config.VmName)' no existe en VirtualBox"
    exit 11
}
```

6.3 Conector equivocado

Durante las pruebas de creación y conexión del disco virtual se presentó un error al momento de asociar el archivo .vdi con la máquina virtual; el disco fue creado correctamente con un tamaño de 4096 MB, pero VirtualBox rechazó la conexión debido al puerto seleccionado por el script.



El mensaje mostrado por VirtualBox indicó que se estaba intentando utilizar el puerto 2, aunque el controlador seleccionado solamente permitía los puertos 0 y 1; por esta razón, la operación storageattach no pudo realizarse y el script registró el mensaje ERROR: fallo al conectar el disco a la VM.

Al revisar el funcionamiento del script, se encontró que el problema estaba en la identificación del controlador utilizado para conectar el disco; primero se comprobaba que existiera un controlador de tipo IntelAhci, correspondiente al controlador SATA, pero posteriormente se obtenía el nombre utilizando siempre storagecontrollername0. Esto podía provocar que se seleccionara otro controlador diferente al SATA, dependiendo de la configuración de la máquina virtual.

Además, el script buscaba un puerto libre comenzando desde el puerto 1; al utilizar el controlador incorrecto y encontrar ocupado el puerto 1, el contador aumentaba hasta 2. Debido a que el controlador seleccionado solamente permitía los puertos 0 y 1, VirtualBox rechazó el valor y generó el error mostrado en la consola.

A partir de este resultado se identificó que el problema no estaba relacionado con la creación del archivo .vdi, sino con la selección del controlador y del puerto utilizado para conectarlo; esto permitió establecer que la lógica encargada de identificar el controlador debía modificarse para utilizar realmente el controlador IntelAhci encontrado durante la validación.

6.4 Error en la ejecución de comandos dentro de la máquina virtual

Durante las pruebas de preparación del disco de datos se presentó un error al ejecutar los comandos de Linux mediante Guest Control; aunque el mecanismo de conexión con la máquina virtual respondió correctamente, al ejecutar el comando utilizado para identificar los discos se obtuvo el mensaje /usr/bin/bash: /usr/bin/bash: cannot execute binary file. Esto impidió obtener correctamente la información generada por lsblk, que era necesaria para identificar el disco de datos.

Al revisar el comportamiento del script, se encontró además un problema en el manejo de esta respuesta; el paso 8.1 obtenía la salida del comando lsblk, pero no verificaba primero si la ejecución había sido exitosa. La función Ejecutar-EnVM sí devuelve un atributo llamado Exitoso para indicar si el comando terminó correctamente, pero este resultado no era validado en dicho paso.

```
[2026-09-27 17:32:25] COMANDO EN VM: sudo -n parted -s /dev//usr/bin/bash: mklabel gpt mkpart primary 0
% 100%
[2026-09-27 17:32:25] RESPUESTA: /usr/bin/bash: /usr/bin/bash: cannot execute binary file
[2026-09-27 17:32:25] ERROR (8.2): fallo al crear la particion
[2026-09-27 17:32:25] El proceso se detuvo por un error en el paso 8.2
[2026-09-27 17:32:26] Tiempos guardados en tiempos.csv
[2026-09-27 17:32:26] Proceso finalizado para la VM: ubuntu
PS C:\Users\mavar\OneDrive\Escritorio\nube>
```

Como consecuencia, el mensaje de error generado por Bash fue interpretado como si correspondiera a la información de un dispositivo; específicamente, el texto /usr/bin/bash: terminó siendo utilizado como parte del nombre del disco, generando posteriormente una ruta incorrecta como /dev//usr/bin/bash:. Esto provocó que el siguiente comando, encargado de crear la partición, también fallará.

El problema permitió identificar dos aspectos a corregir; primero, se debe revisar la forma en que se están ejecutando los comandos mediante Bash dentro de Guest Control, ya que la conexión funciona pero la ejecución mediante bash -c presenta errores. Segundo, el paso 8.1 debe validar el resultado de la ejecución antes de procesar la salida como información del disco, evitando que un mensaje de error sea interpretado como un dispositivo válido. Esta segunda validación permite que el script reporte directamente el fallo real y evita que un error inicial genere errores adicionales en los pasos posteriores.

Durante las pruebas se presentó un error al ejecutar comandos de Linux mediante `Guest Control`; Bash no pudo ejecutar correctamente `lsblk`, por lo que no se obtuvo la información del disco de datos.

Además, el paso 8.1 no verifica si el comando había sido exitoso antes de procesar su salida. Esto provocó que el mensaje de error fuera interpretado como un nombre de disco y generará una ruta incorrecta en el paso 8.2.

La solución consiste en revisar la ejecución de Bash mediante `Guest Control` y validar el atributo `Exitoso` antes de procesar la salida de `lsblk`; de esta forma, el error se identifica directamente y no genera fallos adicionales en los pasos siguientes.

Cambio necesario a realizar:

```
$Salida = & $VBoxManage guestcontrol $Config.VmName run --exe
"/bin/bash" --username $Config.GuestUser --password $Config.Pass -- "-c"
$ComandoFinal 2>&1
```

6.5 Problema con permisos de sudo

Otro problema encontrado se presentó después de corregir la ejecución de los comandos mediante `Guest Control`; en esta ocasión, el comando ya llegaba correctamente a la máquina virtual, pero se generaba un error relacionado con los permisos de `sudo`. El mensaje `sudo: interactive authentication is required` indicaba que el usuario `vboxuser` necesitaba ingresar una contraseña para ejecutar comandos con permisos elevados.

El problema se presentó porque el script utiliza `sudo -n`, lo cual evita que se solicite una contraseña de forma interactiva. Por esta razón, aunque el usuario y la conexión con la máquina virtual funcionaban correctamente, los comandos que necesitaban permisos administrativos no podían ejecutarse.

Como solución, se configuró dentro de la máquina virtual una regla en `sudoers` para permitir que `vboxuser` utilizara `sudo` sin contraseña. Después se verificó la sintaxis de la configuración y se realizó una prueba ejecutando nuevamente el comando `lsblk` desde PowerShell.

Esta modificación permitió que los comandos administrativos pudieran ejecutarse de forma automática mediante `Guest Control`, sin requerir una interacción manual durante el proceso de automatización.

6.6 Problema en identificación del disco

Otro problema encontrado se presentó durante la identificación del disco de datos; aunque `sudo` ya funcionaba correctamente, el paso 8.1 estaba seleccionando un dispositivo incorrecto. La lista obtenida mediante `lsblk` mostraba los discos `sda`, `sdb` y `sdc`, además de la unidad óptica `sr0`. El problema se encontraba en que el código únicamente excluía `sda` y terminaba seleccionando el último dispositivo encontrado, por lo que `sr0` era tomado como si fuera el disco de datos.

Esto provocaba que el siguiente paso intentara crear una partición sobre la unidad óptica; como esta no tenía ningún medio insertado, `parted` generaba el error `No medium found` y el proceso se detenía.

Como solución, se modificó el comando `lsblk` para obtener también el tipo de dispositivo mediante la columna `TYPE`; de esta forma, el código puede filtrar los dispositivos y considerar únicamente aquellos cuyo tipo sea `disk`, descartando unidades ópticas como `sr0` y dispositivos `loop`.

```
2026-09-27 18:16:30] COMANDO EN VM: sudo -n lsblk -n /dev/sr0
2026-09-27 18:16:30] RESPUESTA: sr0  11:0   1 1024M 1 rom
2026-09-27 18:16:30] COMANDO EN VM: sudo -n parted -s /dev/sr0 mklabel gpt mkpart primary 0% 100%
2026-09-27 18:16:30] RESPUESTA: Error: Error opening /dev/sr0: No medium found
2026-09-27 18:16:30] ERROR (8.2): fallo al crear la particion
2026-09-27 18:16:30] El proceso se detuvo por un error en el paso 8.2
2026-09-27 18:16:30] Tiempos guardados en tiempos.csv
2026-09-27 18:16:30] Proceso finalizado para la VM: ubuntu
S C:\Users\mavar\OneDrive\Escritorio\nube>
```

Sin embargo, durante la revisión también se encontraron dos discos de 4 GB, `sdb` y `sdc`, cuando se esperaba únicamente uno. Por esta razón, la corrección del filtro evita seleccionar `sr0`, pero todavía se debe revisar la configuración de la máquina virtual para determinar cuál de los dos discos corresponde al disco de datos y cuál quedó de pruebas anteriores.

Solo se añade esta parte al código:

```
$Resultado81 = Ejecutar-EnVM "lsblk -dn -o NAME,SIZE,TYPE"

# Se recorre linea por linea la respuesta buscando el nombre del
# disco (viene primero en cada linea, por ejemplo: sdb  4G)
$NombreDisco = $null
$NombreDisco = $null
foreach ($Linea in ($Resultado81.Salida -split "`n")) {
    if ($Linea -match "^\s*(\S+)\s+(\S+)\s+(\S+)") {
        $PosibleNombre = $Matches[1]
        $PosibleTipo   = $Matches[3]
        if ($PosibleNombre -ne "sda" -and $PosibleTipo -eq "disk") {
            $NombreDisco = $PosibleNombre
        }
    }
}
```

7. Diseño de Pruebas y Resultados

7.1 Validación de configuración

En este apartado se harán pruebas relacionadas con la validación de la configuración antes de ejecutar comandos sobre la máquina virtual de ubuntu.

7.1 Archivo psd1 no existente

Estado inicial

El archivo vm.psd1 debe encontrarse dentro de la carpeta config, siendo esta la ruta definida por el script para obtener la configuración de la máquina virtual. En el estado inicial de la prueba, se tendrá una copia del archivo para poder retirarla temporalmente sin modificar la configuración original.

Procedimiento

Se va a retirar temporalmente el archivo vm.psd1 de la carpeta config, dejando disponible el resto de archivos necesarios para ejecutar el script. Posteriormente, se ejecutará orquestador.ps1 desde PowerShell y se observará el comportamiento del proceso durante la primera validación.

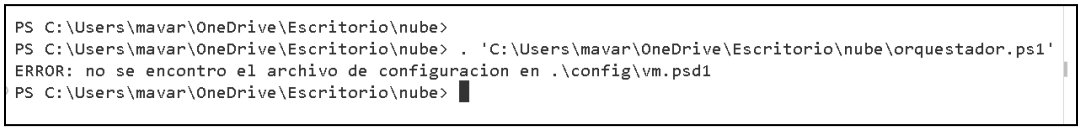
Resultado esperado

El script debe comprobar la existencia de .\config\vm.psd1 en la primera validación; al no encontrarlo, debe mostrar un mensaje que indique que no se halló el archivo de configuración en esa ruta. A continuación, el proceso debe terminar con el código de salida 2. No debe iniciarse ni modificarse ninguna máquina virtual, y no debe crearse ningún disco.

Resultado obtenido

El script detectó que el archivo de configuración no se encontraba en la ruta establecida; por lo tanto, mostró el mensaje indicando que no se encontró el archivo de configuración en .\config\vm.psd1 y finalizó el proceso. Con esto se comprobó que la ejecución no continúa cuando no existe el archivo que contiene los datos necesarios para configurar la máquina virtual.

Evidencia



7.2 Atributo faltante en el psd1

Estado inicial

El archivo vm.psd1 contiene la configuración necesaria para ejecutar el proceso de automatización; para esta prueba se parte de una configuración funcional, pero se elimina

temporalmente el atributo Pass, encargado de almacenar la contraseña del usuario de la máquina virtual. El resto de los atributos se mantiene sin modificaciones.

Procedimiento

Se elimina el atributo Pass del archivo vm.psd1 y posteriormente se ejecuta el script orquestador.ps1; durante la ejecución se observa el comportamiento de las validaciones y el tiempo que tarda el proceso en generar un resultado. Se verifica especialmente si el script identifica de manera directa que falta un atributo requerido.

Resultado esperado

Al leer el archivo de configuración, el script debe verificar que estén presentes los atributos indispensables, entre ellos Pass; al detectar que falta, debe mostrar de inmediato un mensaje que nombre el atributo ausente. El proceso debe detenerse antes de conectar el disco y de encender la máquina virtual. De esta forma, la causa real se informa en pocos segundos y no después de los 300 segundos de espera de Guest Control.

Resultado obtenido

El script no detectó la ausencia del atributo Pass durante la lectura del archivo de configuración; el proceso continuó normalmente y logró iniciar la máquina virtual y obtener su dirección IP, registrando que la VM obtuvo la IP 192.168.20.94 después de aproximadamente 26,87 segundos.

Posteriormente, al intentar comprobar la disponibilidad de Guest Control, el proceso no pudo establecer la conexión; debido a esto, permaneció realizando intentos hasta alcanzar el tiempo máximo de espera de 300 segundos. Finalmente, se mostró el mensaje ERROR: pasó el tiempo máximo y Guest Control nunca respondió (revise usuario/contraseña).

El resultado permitió identificar una deficiencia en la validación inicial de la configuración; aunque el atributo Pass es necesario para utilizar Guest Control, no existe una validación explícita que compruebe que todos los atributos requeridos estén presentes antes de iniciar la máquina virtual. Por esta razón, el error real no se informa inmediatamente y termina

manifestándose varios minutos después mediante un mensaje relacionado con Guest Control, lo cual dificulta identificar que la causa inicial fue la ausencia de la contraseña.

Evidencia

```
VM "ubuntu" has been successfully started.
[2026-09-27 16:59:50] La VM se esta iniciando en modo headless (sin ventana)
[2026-09-27 17:00:12] La VM ya tiene IP: 192.168.20.94 (se demoro 26.8710159 segundo
[2026-09-27 17:05:19] ERROR: paso el tiempo maximo y Guest Control nunca respondio (
rasena)
PS C:\Users\mavar\OneDrive\Escritorio\nube>
```

Solución

Simplemente al momento de obtener los atributos del archivo psd1, se realizará una comprobación de que el usuario y la contraseña están en el archivo; con esto, no volverá a pasar, esperar tanto tiempo para al final recibir un error.

```
# Se comprueba que el archivo de configuracion contenga obligatoriamente
# las claves GuestUser y Pass. Si alguna de las dos falta o esta vacia, el script se deti
if ([string]::IsNullOrWhiteSpace($Config.GuestUser) -or [string]::IsNullOrWhiteSpace($Con
    Write-Host "ERROR: El archivo de configuracion en $RutaConfig no contiene un 'GuestUs
    exit 3 #ERROR_MISSING_CREDENTIALS
}

# ========================================================================
```

7.3 configuración inicial de la VM 1

Estado inicial

La máquina virtual ubuntu se encuentra apagada, no existe el archivo .vdi del disco de datos en el host y la máquina virtual no tiene ningún disco adicional conectado.

Procedimiento

Se ejecuta .\orquestador.ps1 una sola vez y se espera hasta que finalice el proceso. Posteriormente, se revisa el registro generado por la terminal para comprobar que el paso 4 se haya ejecutado correctamente y que los pasos 8.1 a 8.6 hayan finalizado sin errores. Finalmente, se abre el archivo tiempos.csv para verificar que se haya registrado la ejecución con estado OK.

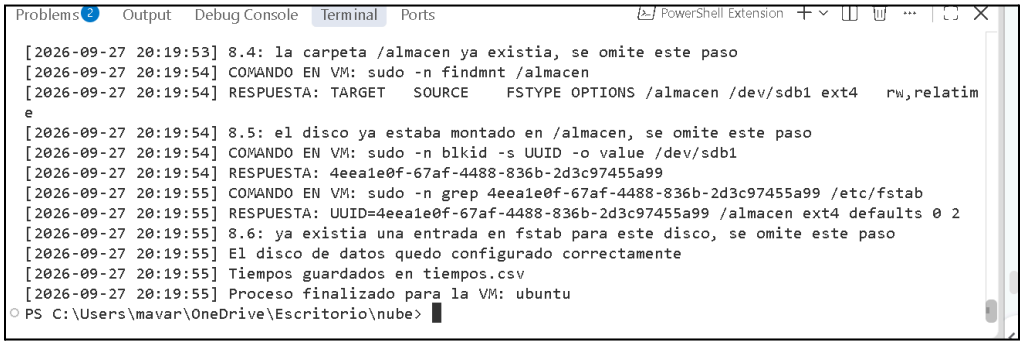
Resultado esperado

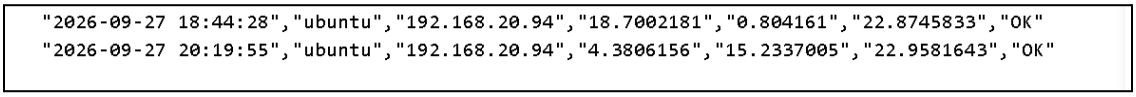
Con la máquina ubuntu apagada y sin disco de datos, el script debe crear el archivo .vdi de 4096 MB y conectarlo al controlador SATA; después debe encender la máquina, esperar la dirección IP y la disponibilidad de Guest Control. Dentro de la VM debe crear la partición y el sistema de archivos ext4, crear la carpeta /almacen y montar el disco. Además, debe agregar la entrada en /etc/fstab. El proceso debe finalizar con el mensaje de configuración correcta, y tiempos.csv debe registrar una fila con estado OK.

Resultado obtenido

La ejecución finaliza correctamente; el script crea y conecta el disco de datos a la máquina virtual, realiza la configuración de la partición, sistema de archivos y punto de montaje, y agrega la entrada correspondiente en /etc/fstab. El log muestra la finalización correcta de los pasos 8.1 a 8.6 y tiempos.csv registra la ejecución con estado OK, confirmando que la configuración inicial de la máquina virtual se realizó correctamente.

Evidencia





Podemos ver que se creó el disco y se registró el tiempo y los datos necesarios.

7.4 Configuración de una segunda VM

Estado inicial

Se dispone de una segunda máquina virtual con las condiciones necesarias para utilizar Guest Control; Guest Additions instaladas, red configurada en modo puente, controlador SATA y un usuario con permisos de sudo sin contraseña. En el archivo config\vm.psd1 se encuentran inicialmente los datos correspondientes a la primera máquina virtual.

Procedimiento

Se reemplazan los valores de VmName, GuestUser, Pass y VdiPath por los correspondientes a la segunda máquina virtual, utilizando un archivo .vdi diferente, disks\debian-srv.vdi. Posteriormente, se ejecuta .\orquestador.ps1 sin agregar parámetros adicionales y se espera hasta que finalice el proceso.

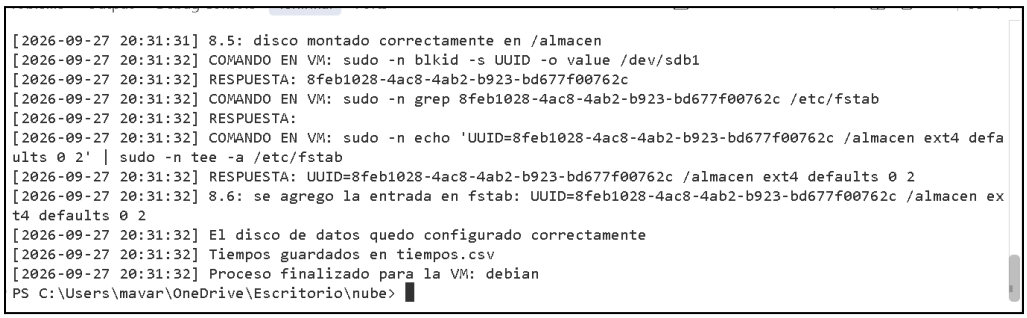
Resultado esperado

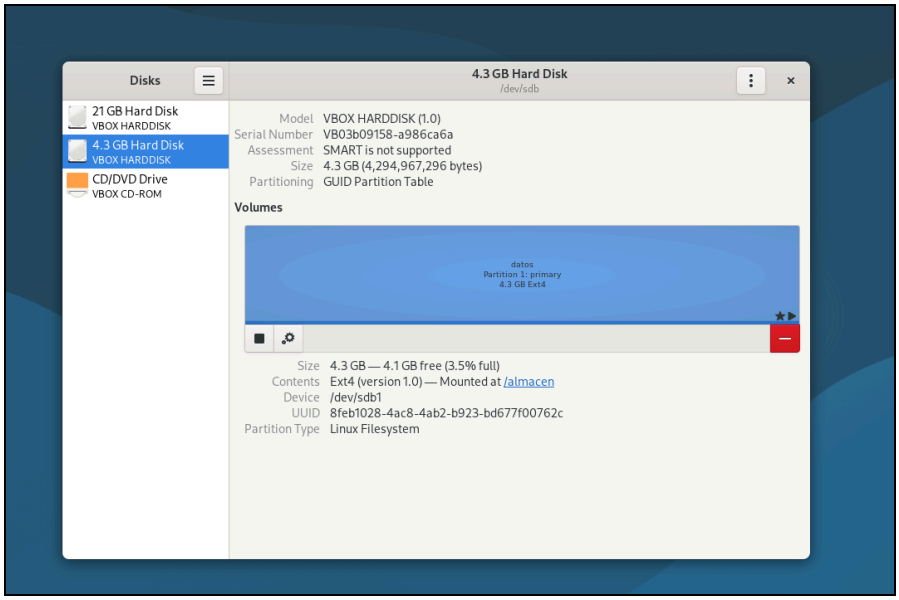
Tras reemplazar en vm.psd1 los valores de VmName, GuestUser, Pass y VdiPath, el script debe configurar la máquina debian sin que se modifique ni se pase ningún parámetro adicional. Debe crear disks\debian-srv.vdi, conectarlo, y ejecutar todos los subpasos internos hasta el final. El proceso debe terminar con estado OK y registrar en tiempos.csv una fila correspondiente a la nueva máquina. Con esto se demuestra que la solución no depende de una VM en particular.

Resultado obtenido

El script utiliza los valores definidos en vm.psd1 para identificar y configurar la segunda máquina virtual; de esta manera, se comprueba que el proceso no depende de una VM específica y que puede reutilizarse para diferentes máquinas siempre que cumplan con los requisitos necesarios para la automatización.

Evidencia





7.5 Disponibilidad del almacenamiento

Estado inicial

La máquina virtual ubuntu fue configurada previamente mediante orquestador.ps1 y se encuentra encendida; el disco de datos de 4 GB quedó particionado, formateado con el sistema de archivos ext4 (formato que define cómo se organizan los archivos dentro del disco) y asociado a la carpeta /almacen. Para esta prueba no se realiza ninguna modificación adicional sobre la máquina; el objetivo es comprobar desde el equipo anfitrión que el almacenamiento está disponible una vez finalizado el proceso.

Procedimiento

Desde el Símbolo del sistema de Windows se emplea Guest Control, mecanismo de VirtualBox que permite ejecutar comandos dentro de la máquina virtual sin abrir una sesión en ella; el nombre de la máquina, el usuario y la contraseña se definen previamente en variables. Se ejecutan tres comandos; findmnt /almacen verifica el punto de montaje (carpeta a través de la cual se accede al disco), df -h /almacen consulta el tamaño y el espacio disponible, y echo prueba | sudo -n tee /almacen/prueba.txt crea un archivo de prueba dentro del disco. La escritura se realiza con sudo porque, por defecto, la carpeta creada por el script pertenece al usuario root; la opción -n evita que el sistema solicite contraseña y detenga el proceso.

Resultado esperado

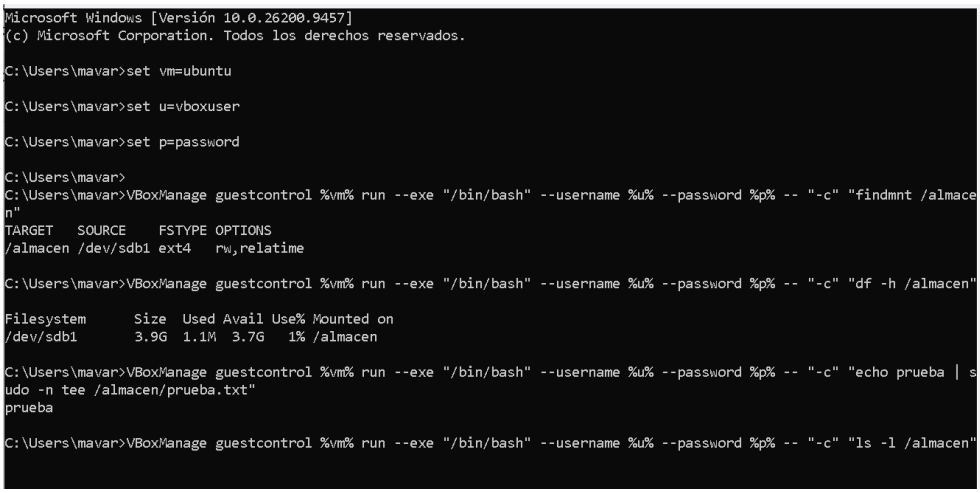
El comando findmnt debe mostrar /almacen montado desde la partición del disco de datos, con sistema de archivos ext4 y permisos de lectura y escritura. El comando df -h debe reportar un tamaño cercano a los 4 GB configurados; la diferencia se debe al espacio que ext4 reserva para su estructura interna. La creación de prueba.txt mediante sudo debe completarse sin errores, lo que confirma que el disco quedó listo para guardar datos.

Resultado obtenido

El comando findmnt mostró /almacen montado desde la partición /dev/sdb1, con sistema de archivos ext4 y permisos de lectura y escritura (rw); el comando df -h reportó un tamaño de 3.9 GB, de los cuales 1.1 MB estaban en uso y 3.7 GB disponibles, equivalentes a un 1 % de ocupación. La diferencia respecto a los 4096 MB configurados se debe al espacio que ext4 reserva para su estructura interna; no corresponde a una pérdida de capacidad útil.

El comando de escritura devolvió el texto prueba sin mensajes de error, lo que confirma que el archivo prueba.txt se creó dentro del disco; la escritura se completó con sudo, ya que la carpeta pertenece a root. Con esto se comprueba que el almacenamiento quedó montado, con la capacidad esperada y disponible para guardar datos; el proceso de automatización cumple así su objetivo de dejar el disco listo para su uso.

Evidencia



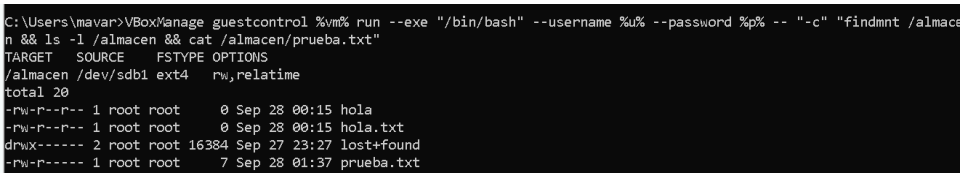
```
total 20
-rw-r--r-- 1 root root     0 Sep 28 00:15 hola
-rw-r--r-- 1 root root     0 Sep 28 00:15 hola.txt
drwx------ 2 root root 16384 Sep 27 23:27 lost+found
-rw-r----- 1 root root     7 Sep 28 01:37 prueba.txt
```

Los archivos hola, se crearon desde la GUI de la máquina virtual, por lo tanto no contamos con ellos.

7.6 Persistencia de los datos tras reiniciar la máquina virtual

Estado inicial

La máquina virtual ubuntu se encuentra en el mismo estado de la prueba anterior; el disco de datos está montado en /almacen y contiene el archivo prueba.txt creado durante esa prueba. La entrada correspondiente está registrada en /etc/fstab, archivo de Linux que indica qué discos se montan automáticamente al arrancar.



Procedimiento

Desde el Símbolo del sistema de Windows, mediante Guest Control, se verifica el contenido de /almacen y se apaga la máquina con sudo -n poweroff. Se confirma el apagado consultando su estado con VBoxManage showvminfo y luego se enciende con VBoxManage startvm, sin ejecutar el script orquestador.ps1. Una vez la máquina termina de arrancar, se consulta de nuevo el contenido de /almacen, incluido el texto de prueba.txt.

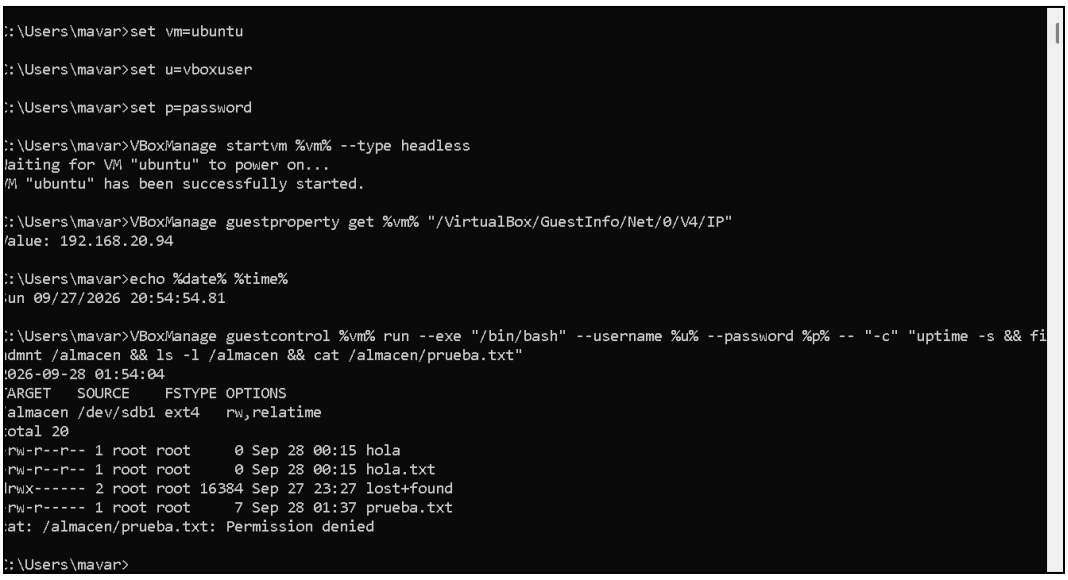
Resultado esperado

Tras el reinicio, el disco debe montarse de forma automática en /almacen, sin intervención del script. Los archivos almacenados antes del apagado, en particular prueba.txt, deben continuar presentes y con su contenido original.

Resultado obtenido

Después de encender la máquina, /almacen apareció montado sobre la partición del disco de datos con sistema de archivos ext4. El archivo prueba.txt continuó disponible y mostró el texto prueba, igual que antes del apagado. Con esto se comprueba que el montaje y los datos se conservan tras un reinicio, y que la entrada de /etc/fstab cumple su función.

Evidencia



7.7 Ejecución repetida sobre una VM ya configurada

Estado inicial

La máquina virtual ubuntu ya fue configurada previamente mediante orquestador.ps1; tiene el disco de datos conectado, con su partición y su sistema de archivos ext4 (formato que define cómo se organizan los archivos dentro del disco). La carpeta /almacen existe, el disco está montado en ella y la entrada correspondiente está registrada en /etc/fstab, archivo de Linux que indica qué discos se montan automáticamente al arrancar. La máquina se encuentra apagada, porque el script exige ese estado.

Procedimiento

Sin realizar ninguna modificación sobre la máquina, se ejecuta nuevamente .\orquestador.ps1 desde PowerShell y se espera a que el proceso finalice. Después se revisa el registro generado para comprobar qué pasos se realizaron y cuáles se omitieron.

Resultado esperado

El script debe detectar que el disco ya está conectado y omitir su creación. Dentro de la máquina, debe reconocer que la partición, el sistema de archivos, la carpeta y el montaje ya existen, y que la entrada de /etc/fstab ya está registrada; por lo tanto, no debe repetir ninguna de esas operaciones. El proceso debe terminar sin errores y sin duplicar la configuración, lo que demuestra la idempotencia, es decir, la capacidad de repetir el proceso sin alterar lo ya hecho.

Resultado obtenido

El registro indicó que el disco de datos ya estaba conectado y omitió el paso 4. Dentro de la máquina, el script identificó el disco como /dev/sdb y detectó que la partición /dev/sdb1 ya existía (paso 8.2), que el sistema de archivos ext4 ya estaba creado (8.3) y que la carpeta /almacen ya existía (8.4). También reconoció que el disco ya estaba montado en /almacen (8.5), gracias a la entrada de fstab procesada al arrancar.

En el paso 8.6, la búsqueda del UUID (identificador único del disco, 4eea1e0f-67af-4488-836b-2d3c97455a99) en /etc/fstab devolvió una sola línea; por lo tanto, la entrada ya existía y no se agregó otra. El proceso finalizó con el mensaje de configuración correcta y guardó la ejecución en tiempos.csv, sin errores ni cambios sobre el disco. Con esto se comprueba que el script puede ejecutarse varias veces sobre la misma máquina sin duplicar configuración ni afectar lo ya existente.

Evidencia

```
[2026-09-27 20:56:47] RESPUESTA: loop0      4K loop loop1   66.8M loop loop2   66.8M loop loop3     20M loop lo
op4   14.2M loop loop5  260.3M loop loop6   16.5M loop loop7  614.5M loop loop8   91.7M loop loop9    1.5M loop
 loop10   402M loop loop11  18.8M loop loop12  18.4M loop loop13  50.1M loop loop14  11.8M loop loop15   828K l
oop sda     30.1G disk sdb        4G disk sr0     1024M rom
[2026-09-27 20:56:47] 8.1: disco de datos identificado como /dev/sdb
[2026-09-27 20:56:47] COMANDO EN VM: sudo -n lsblk -n /dev/sdb
[2026-09-27 20:56:47] RESPUESTA: sdb      8:16  0    4G 0 disk  ÔööÔöÇsdb1   8:17  0    4G 0 part /almacen
[2026-09-27 20:56:47] 8.2: la particion ya existia (/dev/sdb1), se omite este paso
[2026-09-27 20:56:48] COMANDO EN VM: sudo -n blkid /dev/sdb1
[2026-09-27 20:56:48] RESPUESTA: /dev/sdb1: LABEL="datos" UUID="4eea1e0f-67af-4488-836b-2d3c97455a99" BLOCK_SIZ
E="4096" TYPE="ext4" PARTLABEL="primary" PARTUUID="85dddb7b-c32f-42ea-bb42-de86ba4492a4"
[2026-09-27 20:56:48] 8.3: el sistema de archivos ya existia, se omite este paso
[2026-09-27 20:56:48] COMANDO EN VM: sudo -n test -d /almacen && echo EXISTE || echo NOEXISTE
[2026-09-27 20:56:48] RESPUESTA: EXISTE
```

Se conservan los archivos intactos:

```
C:\Users\mavar>VBoxManage guestcontrol %vm% run --exe "/bin/bash" --username %u% --password %p% -- "-c" "uptime -s && fi
ndmnt /almacen && ls -l /almacen && cat /almacen/prueba.txt"
2026-09-28 01:56:33
TARGET   SOURCE    FSTYPE OPTIONS
/almacen /dev/sdb1 ext4   rw,relatime
total 20
-rw-r--r-- 1 root root     0 Sep 28 00:15 hola
-rw-r--r-- 1 root root     0 Sep 28 00:15 hola.txt
drwx------ 2 root root 16384 Sep 27 23:27 lost+found
-rw-r----- 1 root root     7 Sep 28 01:37 prueba.txt
```

7.8 Ejecución con la máquina virtual encendida

Estado inicial

La máquina virtual ubuntu se encuentra encendida. El archivo vm.psd1 contiene una configuración correcta y no se modifica ningún otro elemento del entorno.

Procedimiento

Con la máquina en funcionamiento, se ejecuta .\orquestador.ps1 desde PowerShell y se observa el resultado de las validaciones iniciales. Después se revisa que la máquina y el disco no hayan sufrido cambios.

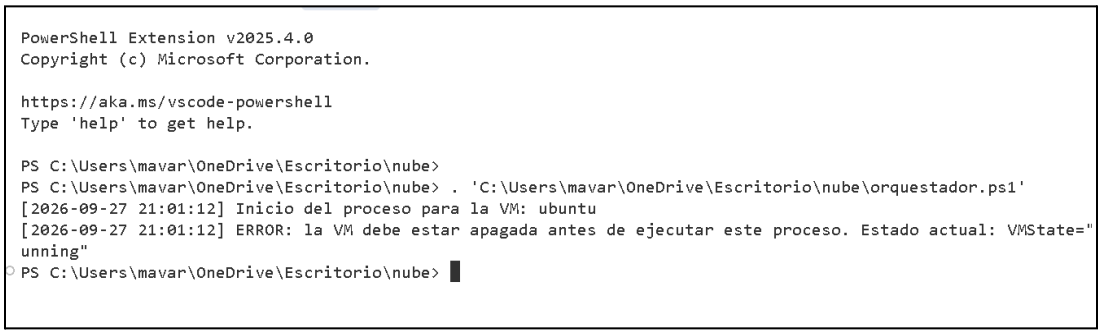
Resultado esperado

El script debe comprobar el estado de la máquina antes de tocar el disco. Al detectar que no está apagada, debe mostrar y registrar un mensaje que indique el motivo junto con el estado actual, y terminar con un código de error. No debe crear ni conectar ningún disco, ni apagar o reiniciar la máquina.

Resultado obtenido

El script pasó la lectura de la configuración, detectó que la máquina estaba en ejecución y mostró el mensaje "la VM debe estar apagada antes de ejecutar este proceso", acompañado del estado actual (VMState="running"). El proceso se detuvo en esa validación, el 27 de septiembre de 2026 a las 21:01:12, sin llegar a los pasos de creación y conexión del disco ni al encendido de la máquina. La misma respuesta se había observado antes, a las 18:41:58 y a las 18:43:55. Con esto se comprueba que el script se protege ante una condición que impide su ejecución normal, y que informa la causa de forma inmediata y clara.

Evidencia



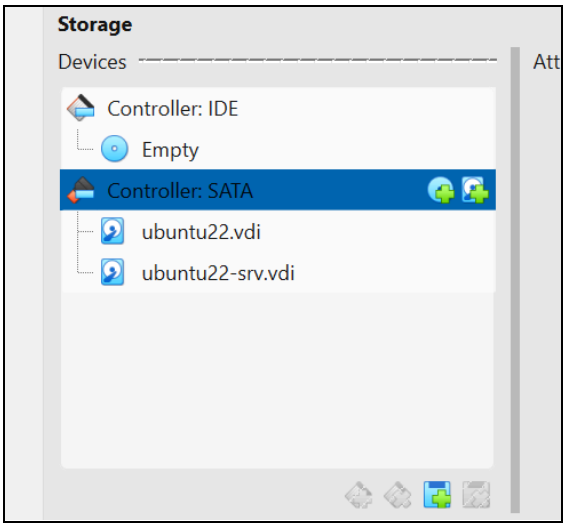
8. Evidencias ejecución correcta

Luego de todas las comprobaciones, pruebas y correcciones, se muestran los resultados de todo el script. Para empezar se procede a mostrar todos los resultados que nos proporciona la terminal:

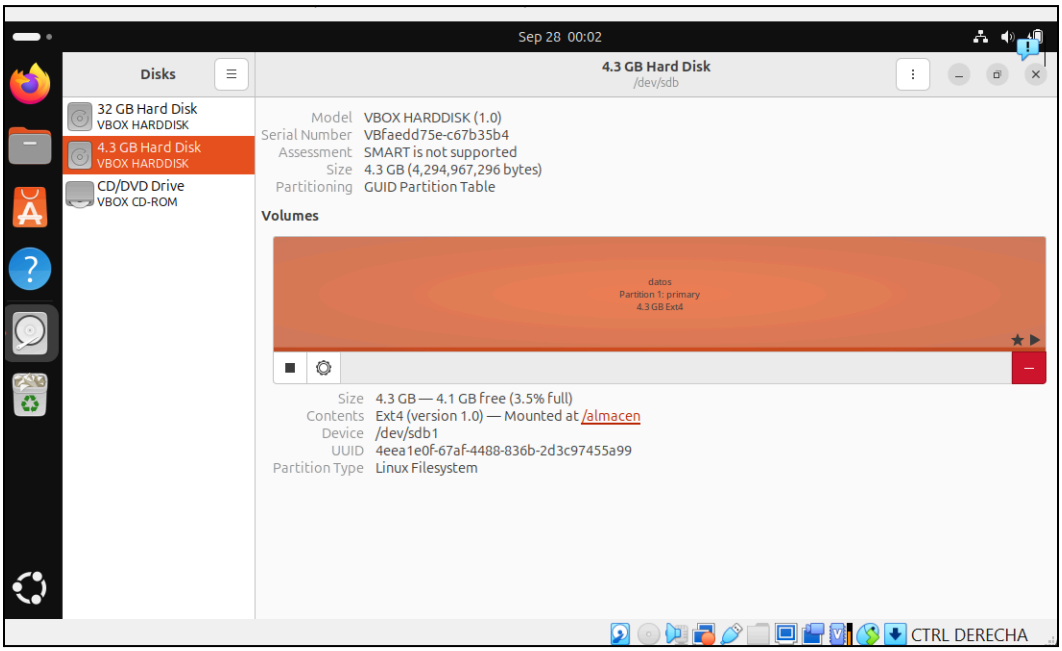
```
PS C:\Users\mavar\OneDrive\Escritorio\nube> .
'C:\Users\mavar\OneDrive\Escritorio\nube\orquestador.ps1'
[2026-09-27 18:44:05] Inicio del proceso para la VM: ubuntu
[2026-09-27 18:44:05] Todas las validaciones fueron superadas
[2026-09-27 18:44:05] El disco de datos ya estaba conectado a la VM, se omite este paso
Waiting for VM "ubuntu" to power on...
VM "ubuntu" has been successfully started.
[2026-09-27 18:44:09] La VM se esta iniciando en modo headless (sin ventana)
[2026-09-27 18:44:24] La VM ya tiene IP: 192.168.20.94 (se demoro 18.7002181 segundos)
[2026-09-27 18:44:24] Guest Control ya esta disponible (se demoro 0.804161 segundos)
[2026-09-27 18:44:25] COMANDO EN VM: sudo -n lsblk -dn -o NAME,SIZE,TYPE
[2026-09-27 18:44:25] RESPUESTA: loop0      4K loop loop1   66.8M loop loop2     20M loop
loop3   14.2M loop loop4   16.5M loop loop5  260.3M loop loop6  614.5M loop loop7   91.7M
loop loop8    1.5M loop loop9    402M loop loop10  18.4M loop loop11  18.8M loop loop12
11.8M loop loop13  50.1M loop loop14   828K loop sda     30.1G disk sdb        4G disk sdc
```

```
4G disk sr0     1024M rom
[2026-09-27 18:44:25] 8.1: disco de datos identificado como /dev/sdc
[2026-09-27 18:44:25] COMANDO EN VM: sudo -n lsblk -n /dev/sdc
[2026-09-27 18:44:25] RESPUESTA: sdc      8:32  0    4G 0 disk  ÔööÔöÇsdc1   8:33  0    4G
0 part
[2026-09-27 18:44:25] 8.2: la particion ya existia (/dev/sdc1), se omite este paso
[2026-09-27 18:44:25] COMANDO EN VM: sudo -n blkid /dev/sdc1
[2026-09-27 18:44:25] RESPUESTA: /dev/sdc1: LABEL="datos"
UUID="4eea1e0f-67af-4488-836b-2d3c97455a99" BLOCK_SIZE="4096" TYPE="ext4"
PARTLABEL="primary" PARTUUID="85dddb7b-c32f-42ea-bb42-de86ba4492a4"
[2026-09-27 18:44:25] 8.3: el sistema de archivos ya existia, se omite este paso
[2026-09-27 18:44:26] COMANDO EN VM: sudo -n test -d /almacen && echo EXISTE || echo
NOEXISTE
[2026-09-27 18:44:26] RESPUESTA: EXISTE
[2026-09-27 18:44:26] 8.4: la carpeta /almacen ya existia, se omite este paso
[2026-09-27 18:44:26] COMANDO EN VM: sudo -n findmnt /almacen
[2026-09-27 18:44:26] RESPUESTA:
[2026-09-27 18:44:27] COMANDO EN VM: sudo -n mount /dev/sdc1 /almacen
[2026-09-27 18:44:27] RESPUESTA:
[2026-09-27 18:44:27] 8.5: disco montado correctamente en /almacen
[2026-09-27 18:44:27] COMANDO EN VM: sudo -n blkid -s UUID -o value /dev/sdc1
[2026-09-27 18:44:27] RESPUESTA: 4eea1e0f-67af-4488-836b-2d3c97455a99
[2026-09-27 18:44:27] COMANDO EN VM: sudo -n grep
4eea1e0f-67af-4488-836b-2d3c97455a99 /etc/fstab
[2026-09-27 18:44:27] RESPUESTA:
[2026-09-27 18:44:28] COMANDO EN VM: sudo -n echo
'UUID=4eea1e0f-67af-4488-836b-2d3c97455a99 /almacen ext4 defaults 0 2' | sudo -n tee -a
/etc/fstab
[2026-09-27 18:44:28] RESPUESTA: UUID=4eea1e0f-67af-4488-836b-2d3c97455a99
/almacen ext4 defaults 0 2
[2026-09-27 18:44:28] 8.6: se agrego la entrada en fstab:
UUID=4eea1e0f-67af-4488-836b-2d3c97455a99 /almacen ext4 defaults 0 2
[2026-09-27 18:44:28] El disco de datos quedo configurado correctamente
[2026-09-27 18:44:28] Tiempos guardados en tiempos.csv
[2026-09-27 18:44:28] Proceso finalizado para la VM: ubuntu
PS C:\Users\mavar\OneDrive\Escritorio\nube>
```

Podemos ver que al menos la terminal indica que todo quedó correctamente montado y configurado; sin embargo, puede no ser suficiente para demostrar el funcionamiento, ahora miramos en los discos del sistema que se haya creado correctamente y se le haya asignado correctamente a la máquina virtual en este caso ‘ubuntu’.



Se puede observar que se crea el nuevo disco llamado ubuntu-srv.vdi y se le asigna al controlador SATA de la máquina virtual. Se continúa encendiendo la interfaz visual de la máquina virtual para observar los cambios, se procede a entrar a la aplicación de discos y podemos observar el nuevo disco configurado



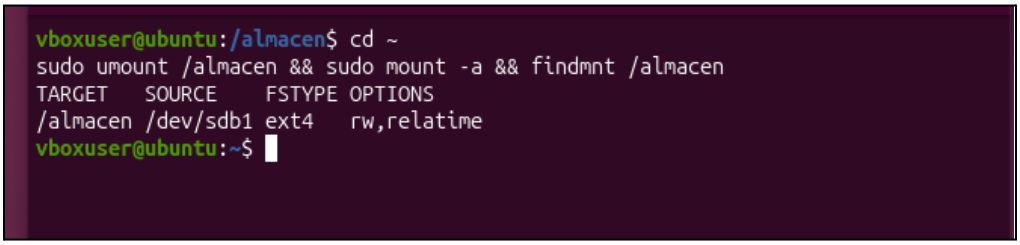
Como comprobación adicional, se procede a desmontar manualmente el punto de montaje /almacen y posteriormente se ejecuta mount -a, comando que permite volver a montar los sistemas de archivos definidos en /etc/fstab. Finalmente, mediante findmnt /almacen se verifica dónde quedó montado nuevamente el disco.

El resultado obtenido fue:

TARGET SOURCE FSTYPE OPTIONS
/almacen /dev/sdc1 ext4 rw,relatime

Este resultado confirma que la partición /dev/sdc1 se montó nuevamente en /almacen utilizando el sistema de archivos ext4. Además, demuestra que la entrada agregada en /etc/fstab funciona correctamente, ya que el montaje pudo realizarse mediante mount -a sin tener que indicar manualmente el dispositivo o la ruta.

De esta manera, no solamente se comprueba que el script creó y montó correctamente el disco, sino también que la configuración realizada permite recuperar el montaje después de desmontarlo; esto evidencia que el almacenamiento quedó configurado para utilizarse de forma persistente en la máquina virtual.



Tiempos de ejecución de algunas pruebas:

Se adjunta una tabla con algunos casos de ejecución exitosos para comparar los tiempos entre las dos máquinas, ubuntu y debian; se evidencia que en ubuntu es más rápido.

#	Fecha y hora	VM	IP	Hasta IP (s)	IP → Guest Control (s)	Hasta disponibilidad (s)	Total (s)	Estado
1	2026-09-27 18:44:28	ubuntu	192.168.20.94	18.70	0.80	19.50	22.87	OK
2	2026-09-27 20:19:55	ubuntu	192.168.20.94	4.38	15.23	19.61	22.96	OK

3	2026-09-27 20:31:32	debian	192.168.20.95	31.00	0.79	31.78	34.86	OK
4	2026-09-27 20:56:50	ubuntu	192.168.20.94	4.18	15.74	19.92	23.27	OK

Herramientas de IA

Herramienta	Uso en el proyecto
Claude (Anthropic)	Depuración del código del script orquestador.ps1.
Gemini (Google)	Creación del diagrama del escenario de aplicación (punto 5).

Parámetros de la máquinas virtuales

Parámetro	ubuntu	debian	¿Cambió?	Función
VmName	ubuntu	debian	Sí (obligatorio)	Identifica la VM en VirtualBox
VdiPath	disks\ubuntu-srv.vdi	disks\debian-srv.vdi	Sí (obligatorio)	Archivo .vdi del disco de datos en el host; debe ser único por VM
GuestUser	vboxuser	vboxuser	No	Usuario de Linux para Guest Control
Pass	(no se muestra)	(no se muestra)	No	Contraseña del usuario invitado
DiskSizeMb	4096	4096	No	Tamaño del disco de datos
MountPoint	/almacen	/almacen	No	Carpeta de montaje dentro de la VM
FsLabel	datos	datos	No	Etiqueta del sistema de archivos
UseSudo	$true	$true	No	Anteponer sudo -n a los comandos

9. Referencias

Free Software Foundation. (s. f.). GNU Parted user manual. https://www.gnu.org/software/parted/manual/parted.html

Microsoft. (s. f.). Import-PowerShellDataFile. Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/import-powershelldatafile

Oracle. (s. f.). Oracle VM VirtualBox user guide for release 7.2. https://www.virtualbox.org/manual

Red Hat. (s. f.). What is infrastructure as code (IaC)? https://www.redhat.com/en/topics/automation/what-is-infrastructure-as-code-iac

util-linux developers. (s. f.). blkid(8): Linux manual page. man7.org. https://man7.org/linux/man-pages/man8/blkid.8.html

util-linux developers. (s. f.). findmnt(8): Linux manual page. man7.org. https://man7.org/linux/man-pages/man8/findmnt.8.html

util-linux developers. (s. f.). fstab(5): Linux manual page. man7.org. https://man7.org/linux/man-pages/man5/fstab.5.html

util-linux developers. (s. f.). lsblk(8): Linux manual page. man7.org. https://man7.org/linux/man-pages/man8/lsblk.8.html